
AN0040-EC NB-IoT OpenCPU 软件开发指南_V1.3

EigenCOMM Wireless Microcontroller

文档说明

本文档用于帮助用户如何将移芯 NB-IoT 芯片作为主控处理器芯片进行二次开发，用户可以直接在芯片里开发集成应用，省掉外部的 MCU，从而节省硬件成本。通过本文档，开发人员能够快速了解 OpenCPU 能够提供的解决方案，并能够快速掌握 OpenCPU 的整个软件开发流程，从而大大缩短产品的评估和开发周期。

目录

1. 引言	5
1.1 文档目的	5
1.2 总体开发流程	5
2. OpenCPU 综述	5
2.1 总体架构	5
2.2 OpenCPU 开放资源	5
2.2.1 处理器	5
2.2.2 内存	6
2.3 OpenCPU 编译说明	6
2.4 OpenCPU 数据类型说明	6
2.5 OpenCPU 使用限制	6
3. OpenCPU API 函数接口	7
3.1 System API	7
3.1.1 拷贝字符串副本	7
3.1.2 assert 系统调用	7
3.1.3 OSA signal 机制	7
3.1.4 动态内存分配释放	8
3.2 Driver API	8
3.3 Log API	8
3.3.1 设置 unilog 打印级别	9
3.3.2 Unilog 通用打印接口	9
3.3.3 Unilog 普通 dump 数据接口	9
3.3.4 Unilog 增强 dump 数据接口	10
3.3.5 Unilog 字符串打印接口	10
3.4 Power API	10
3.5 Filesystem API	11
3.5.1 创建/打开文件	11
3.5.2 读取文件内容	11
3.5.3 写入文件内容	11
3.5.4 同步文件内容	12
3.5.5 关闭文件	12
3.5.6 删除文件	12
3.5.7 设置文件指针	12
3.5.8 获取文件指针	13
3.5.9 获取文件大小	13
3.6 Socket API	13
3.6.1 创建 socket	13
3.6.2 设置 socket 属性参数	14
3.6.3 socket 连接	14

3.6.4	socket 发送数据	14
3.6.5	socket 接收数据	15
3.6.6	select 监控数据	15
3.6.7	关闭 socket	15
3.6.8	TCP 带 RAI 功能的 socket 发送操作	16
3.6.9	UDP 带 RAI 功能的 socket 发送操作	16
3.7	Protocol Stack API	17
3.7.1	注册 PS 事件 callback	17
3.7.2	注销 PS 事件注册	18
3.7.3	触发 PS 事件	18
3.7.4	获取 IMSI 号	18
3.7.5	获取 ICCID 号	19
3.7.6	获取 IMEI 号	19
3.7.7	获取 SN 号	19
3.7.8	获取激活状态 CID 信息	19
3.7.9	获取终端 PSM 配置信息	20
3.7.10	获取网络侧下发的 TAU 信息	20
3.7.11	修改终端 PSM 配置信息	20
3.7.12	获取终端注网状态	21
3.7.13	获取终端网络信息	21
3.7.14	获取基站位置信息	23
3.7.15	获取终端 EDRX 信息	23
3.7.16	配置终端 EDRX 信息	23
3.7.17	获取搜网 PowerLevel 信息	24
3.7.18	获取 APN 配置信息	24
3.7.19	检查是否已经同步时间	24
3.7.20	获取当前时间	25
3.7.21	获取当前 UTC 时间	25
3.7.22	设置当前时间	25
3.7.23	获取终端覆盖等级信息	26
3.7.24	获取终端信号质量信息	26
3.7.25	获取终端版本信息	26
3.7.26	软复位终端 NB 功能	27
3.7.27	设置启动时 NB 默认功能模式	27
3.7.28	获取启动时 NB 默认功能模式	27
3.7.29	打开 SIM 逻辑通道	27
3.7.30	关闭 SIM 逻辑通道	28
3.7.31	SIM 逻辑通道通用访问	28
3.7.32	SIM 通用访问	28
3.7.33	BAND 设置	29
3.7.34	获取当前 BAND 设置	29
3.7.35	获取支持的 BAND	29
3.7.36	获取服务小区和邻区基本信息	30

3.7.37	设置锁频，锁小区	31
3.7.38	查询锁频，锁小区状态	32
4.	附录	32
4.1	上电启动流程	32
4.2	OpenCPU 应用示例	34
5.	版本	35
6.	关于移芯通信	错误!未定义书签。

1. 引言

1.1 文档目的

移芯 NB-IoT 芯片除了可以作为标准的 AT 指令模块与外部主 MCU 一起工作外，还可以作为 OpenCPU 主控处理器使用。本文将详细介绍 OpenCPU 的实现，从开放的硬件资源入手，逐一介绍 SDK 中每个模块能够提供的功能，以及提供给开发人员调用的软件 API 接口。

1.2 总体开发流程

- 1) 获取 Eigencomm SDK
- 2) 熟悉 OpenCPU 的整体框架
- 3) 根据产品需求提出解决方案
- 4) 评估并实现方案

2. OpenCPU 综述

2.1 总体架构



图 1 OpenCPU 总体架构

2.2 OpenCPU 开放资源

2.2.1 处理器

32 位 ARM Cortex-M3 处理器 工作频率 204MHz

2.2.2 内存

- 内部集成有 256KB+16KB SRAM 与 4MB NorFlash

其中提供给用户进行 APP 开发的内存资源为：

- 1MB Flash 空间
- SRAM：将 RW/ZI 数据段中剩余部分的空间全部分配给 HEAP，即 HEAP size 会根据 RW/ZI 段的大小动态变化，scatter 文件中加入检查，HEAP 至少维持 56KB. 客户可以直接定义自己的全局变量，但应尽量限制全局变量的大小，合理使用内存。

2.3 OpenCPU 编译说明

参考 EC NB-IoT SDK 编译指南中的“定制化编译的使用”章节，在工程的 Makefile 文件里修改如下，表示不编译 AT 相关的模块：

```
AVAILABLE_TARGETS = ec616_0h00 ec616z0_0h00
TOOLCHAIN          = ARMCC
BINNAME            = onenet-demo-flash
TOP                := ../../../../../../../..

BUILD_AT           = n
BUILD_AT_DEBUG     = n
THIRDPARTY_CISONENET_ENABLE = y
THIRDPARTY_WAKAAMA_ENABLE = n

CFLAGS_INC         += -I ../inc
obj-y              += PLAT/project/$(TARGET)/apps/onenet_example/src/app.o \
                     PLAT/project/$(TARGET)/apps/onenet_example/src/cisAsynEntry.o \
                     PLAT/project/$(TARGET)/apps/onenet_example/src/bsp_custom.o

include $(TOP)/PLAT/tools/scripts/Makefile.rules
```

图 2 OpenCPU 编译说明

2.4 OpenCPU 数据类型说明

建议统一使用 commontypedef.h 中定义的基础数据类型风格。在源代码编写过程中可以通过 #include “commontypedef.h” 包含进来。

2.5 OpenCPU 使用限制

1. 中断相关执行时间限制
单个中断执行时间需要小于 50 微秒，1 毫秒内累计关中断时间小于 100 微秒
2. Task 任务优先级限制
所有应用级别的 TASK 优先级必须小于等于 osPriorityNormal(24)
3. 开启 PMU 功能后，如果需要使用一直连续不中断的 timer，只能使用 HIB timer API，其他情况下，可以考虑使用 OS timer API，这两个接口的区别主要在于深度睡眠的情况下计时操作是否会继续保持，用户可以根据自己的需求选择使用合适的 API。

3. OpenCPU API 函数接口

3.1 System API

SDK 中提供了两种类型的系统调用接口：CMSIS-OS2 类型和移芯自定义的 OS 适配层，对于 CMSIS-OS2 类型，用户可以自行参考 FreeRTOS CMSIS-OS2 相关的 API 文档来使用，本章节主要介绍一些 SDK 二次开发过程中使用频率较高的一些 API 接口。

3.1.1 拷贝字符串副本

- 头文件

```
#include "osasys.h"
```

- 原型

```
CHAR* strdup(const CHAR*string)
```

- 参数

string: 目标字符串

- 返回值

返回新的字符串的副本

- 说明

此接口实现了一个类似 POSIX 风格的字符串拷贝函数，在移植第三方开源代码中，通常会用到这个函数。

3.1.2 assert 系统调用

- 头文件

```
#include "os_exception.h"或者#include "assert.h"
```

- 原型

```
EC_ASSERT(x,v1,v2,v3)  
assert(x)
```

- 参数

x: 抛出 ASSERT 的条件，FALSE 触发

v/v2/v3: ASSERT 时可以打印出来的三个参数

- 返回值

N/A

- 说明

SDK 提供两种系统 assert 接口，一种是 EC_ASSERT()，需要包含" os_exception.h"，此接口会将出错时的 ARM 寄存器以及运行上下文信息通过 UNILOG 打印出来，还会根据需求保存到 FLASH 中供后续定位使用；另外一种简单版本的 assert()，需要包含" assert.h"，在移植开源代码的时候，很多代码已经大量使用了编译器自带的 assert() 接口，此时可以使用简单版本的 assert()。

3.1.3 OSA signal 机制

SDK 中移芯自定义 OS 适配层提供了几种 signal 接口供用户使用。

创建 signal 的 API 接口有：

```
void OsaCreateSignal (UINT16 sigId, UINT16 sigBodySize, SignalBuf **signal)
```

```
void OsaCreateZeroSignal (UINT16 sigId, UINT16 sigBodySize, SignalBuf **signal)
```

```
void OsaCreateFastSignal (UINT16 sigId, UINT16 sigBodySize, SignalBuf **signal)
```

OsaCreateSignal()与 OsaCreateZeroSignal()的区别就是创建 sigBody 是否进行了清零操作，通常如果创建的 signal 数据结构明确知道在接下来的代码中会全部进行赋值，可以考虑使用 OsaCreateSignal()。另外由于 OsaCreateSignal()和 OsaCreateZeroSignal()里都使用了 malloc 接口，所以不能使用在中断上下文里，如果要在中断上下文里使用 signal 机制，可以调用 OsaCreateFastSignal()。

发送 signal 的 API 接口有：

```
void OsaSendSignal(UINT16 taskId, SignalBuf **signal)
```

```
void OsaSendDumpSignal(UINT16 taskId, SignalBuf **signal)
```

```
void OsaSendNoDumpSignal(UINT16 taskId, SignalBuf **signal)
```

三者的区别是 OsaSendSignal()只发送 signal，不 dump 和 record signal；OsaSendDumpSignal()发送 signal 的同时，dump 该 signal 的内容，但是不做 record；OsaSendNoDumpSignal()发送 signal 的同时，record 该 signal，但是不 dump 该 signal。

Dump 和 record 都是 debug 该 signal 的方式，主要体现在 unilog 的打印上。

接收 signal 的 API 接口只有一个：

```
void OsaReceiveSignal(UINT16 taskId, SignalBuf **signal)
```

销毁 signal 的 API 接口有：

```
void OsaDestroySignal (SignalBuf **signal)
```

```
void OsaDestroyFastSignal (SignalBuf **signal)
```

OsaDestroySignal()用于销毁 OsaCreateSignal()和 OsaCreateZeroSignal()创建的 signal；OsaDestroyFastSignal()用于销毁 OsaCreateFastSignal()创建的 signal。

3.1.4 动态内存分配释放

- 头文件

```
#include "cmsis_os2.h"
```

- 原型

```
void* malloc(size_t size)
```

```
void* realloc(void *pv, size_t xWantedSize)
```

```
void* calloc(size_t n, size_t Size)
```

```
void* free(void *p)
```

- 参数

N/A

- 返回值

申请内存操作时，返回 NULL 表示申请内存失败；不为 NULL 表示申请到的内存地址指针

3.2 Driver API

该章节请参考 SDK 中专门介绍驱动 API 接口的文档“EC61x_API_Reference_Manual”。

3.3 Log API

SDK 支持 printf, Segger_RTT 打印 LOG 的方式，但是不建议用户使用，因为它们打印效率低，在 NB 通讯过程中会严重

影响调度的实时性。推荐使用 EC 的高速 LOG 接口 UNILog，该接口通过 SOC 内部硬件加速实现低延迟高效率的 LOG 打印，该接口通过高速串口将 LOG 发送到 PC 上，用户可以通过专用的 LOG 查看工具 EPAT 离线或者实时查看 LOG。本章节主要介绍如何使用 UNILog API 接口。

3.3.1 设置 unilog 打印级别

- 头文件

```
#include "debug_trace.h"
```

- 原型

```
void uniLogDebugLevelSet (debugTraceLevelType debugLevel)
```

- 参数

debugLevel: LOG 的打印级别

支持 6 个打印级别 P_DEBUG/P_INFO/P_VALUE/P_SIG/P_WARNING/P_ERROR

- 返回值

N/A

3.3.2 Unilog 通用打印接口

- 头文件

```
#include "debug_trace.h"
```

- 原型

```
ECOMM_TRACE(moduleID, subID, debugLevel, argLen, format, ...)
```

- 参数

moduleID: LOG 模块 ID 信息

subID: 相同 moduleID 下的 subID 信息,用于区分相同模块下不同的函数内的打印

debugLevel: 打印级别信息

argLen: 打印参数的个数

format: 打印的格式化输出内容

... : 不定长打印输入参数

注: 目前支持的格式为: %d,%x,%f,%u。

- 返回值

N/A

3.3.3 Unilog 普通 dump 数据接口

- 头文件

```
#include "debug_trace.h"
```

- 原型

```
ECOMM_DUMP(moduleID, subID, debugLevel, format, dumpLen, dump)
```

- 参数

moduleID: LOG 模块 ID 信息

subID: 相同 moduleID 下的 subID 信息,用于区分相同模块下不同的函数内的打印

debugLevel: 打印级别信息

format: 打印的格式化输出内容

dumpLen: dump 数据的长度

dump: dump 数据的内容

- 返回值

N/A

注:不建议用户打印太长的数据,如超过 200byte,大概率会出现 log 丢失。

3.3.4 Unilog 增强 dump 数据接口

- 头文件

```
#include "debug_trace.h"
```

- 原型

```
ECOMM_DUMP_POLLING(moduleID, subID, debugLevel, format, dumpLen, dump)
```

- 参数

moduleID: LOG 模块 ID 信息

subID: 相同 moduleID 下的 subID 信息,用于区分相同模块下不同的函数内的打印

debugLevel: 打印级别信息

format: 打印的格式化输出内容

dumpLen: dump 数据的长度

dump: dump 数据的内容

- 返回值

N/A

该接口会先去查询 UNILOG 硬件的缓冲是否空闲,然后根据结果去发送,避免直接发送引起的部分场景丢 LOG 的问题。

但不建议用户打印太长的数据,会造成调用 task 被 delay。

3.3.5 Unilog 字符串打印接口

- 头文件

```
#include "debug_trace.h"
```

- 原型

```
ECOMM_STRING(moduleID, subID, debugLevel, format, string)
```

- 参数

moduleID: LOG 模块 ID 信息

subID: 相同 moduleID 下的 subID 信息,用于区分相同模块下不同的函数内的打印

debugLevel: 打印级别信息

format: 打印的格式化输出内容

string: 待打印的字符串信息

- 返回值

N/A

3.4 Power API

该章节请参考 SDK 专门介绍低功耗 API 接口的文档“AN0024-EC NB-IoT PMU 低功耗开发手册_V1.0.pdf”。

3.5 Filesystem API

SDK 中提供了 littlefs 文件系统，用户可以使用相关的 API 接口来进行创建/读取/写入/删除操作。

3.5.1 创建/打开文件

- 头文件

```
#include "osasys.h"
```

- 原型

```
OSAFILE OsaFopen(const char *fileName, const char* mode)
```

- 参数

fileName: 文件名

mode: 文件访问模式字符串，支持的模式如下：

"r" or "rb"	只读，如果文件不存在，打开失败
"w"	只写，如果文件不存在，打开失败
"r+" or "rb+"	读写，如果文件不存在，打开失败
"wb"	写，如果文件不存在则创建
"wb+"	读写，如果文件不存在则创建

- 返回值

如果打开成功，返回后续文件操作的文件句柄，失败返回 NULL

3.5.2 读取文件内容

- 头文件

```
#include "osasys.h"
```

- 原型

```
UINT32 OsaFread(void *buf, UINT32 size, UINT32 count, OSAFILE fp)
```

- 参数

buf: 指向最小尺寸 size*count 字节的内存块指针

size: 要读取的每个元素的大小，以字节为单位

count: 要读取的元素个数，每个元素的大小为 size 字节

fp: 文件句柄，由 OsaFopen 函数返回得到

- 返回值

成功读取的元素总数

3.5.3 写入文件内容

- 头文件

```
#include "osasys.h"
```

- 原型

```
UINT32 OsaFwrite(void *buf, UINT32 size, UINT32 count, OSAFILE fp)
```

- 参数

buf: 指向要被写入的元素数组的指针

size: 要写入的每个元素的大小，以字节为单位

count: 要写入的元素个数，每个元素的大小为 size 字节

fp: 文件句柄, 由 OsaFopen 函数返回得到

- 返回值

成功写入的元素总数

3.5.4 同步文件内容

- 头文件

```
#include "osasys.h"
```

- 原型

```
INT32 OsaFsync(OSAFILE fp)
```

- 参数

fp: 文件句柄, 由 OsaFopen 函数返回得到

- 返回值

等于 0 表示操作成功, -1 表示失败

3.5.5 关闭文件

- 头文件

```
#include "osasys.h"
```

- 原型

```
INT32 OsaFclose(OSAFILE fp)
```

- 参数

fp: 将要关闭的文件句柄, 由 OsaFopen 函数返回得到

- 返回值

0

3.5.6 删除文件

- 头文件

```
#include "osasys.h"
```

- 原型

```
UINT32 OsaFremove(const char *fileName)
```

- 参数

fileName: 待删除的文件名

- 返回值

0

3.5.7 设置文件指针

- 头文件

```
#include "osasys.h"
```

- 原型

```
INT32 OsaFseek(OSAFILE fp, INT32 offset, UINT8 seekType)
```

- 参数

fp: 文件句柄, 由 OsaFopen 函数返回得到

offset: 文件指针相对 seekType 的偏移, 以字节为单位

seekType: 表示开始加偏移 offset 的位置。有效值为:

SEEK_SET	文件开头
SEEK_CUR	文件指针当前位置
SEEK_END	文件末尾

- 返回值
-1 表示设置文件指针失败，等于 0 表示设置成功

3.5.8 获取文件指针

- 头文件
`#include "osasys.h"`
- 原型
`INT32 OsaFtell(OSAFILE fp)`
- 参数
fp: 文件句柄，由 OsaFopen 函数返回得到
- 返回值
大于 0 表示当前文件指针位置，-1 表示失败

3.5.9 获取文件大小

- 头文件
`#include "osasys.h"`
- 原型
`INT32 OsaFsize(OSAFILE fp)`
- 参数
fp: 文件句柄，由 OsaFopen 函数返回得到
- 返回值
大于 0 表示文件大小，-1 表示失败

3.6 Socket API

SDK 中 lwip 软件是完全开放的，所以用户可以直接使用 lwip 中输出的 Socket 相关的 API，这里只罗列部分常用 API，实际使用过程中，也可以根据实际需求直接调用 lwip 其他 API 接口。

3.6.1 创建 socket

- 头文件
`#include "lwip/sockets.h"`
- 原型
`int socket(int domain, int type, int protocol)`
- 参数
domain: AF_INET
type: 支持三种 socket 协议类型 SOCK_STREAM/SOCK_DGRAM/SOCK_RAW，分别对应 TCP/UDP/RAW
protocol: 如果是 TCP，此值为 IPPROTO_TCP；如果是 UDP，此值为 IPPROTO_UDP
- 返回值

小于 0 表示创建 socket 失败，否则为 socket 句柄

3.6.2 设置 socket 属性参数

- 头文件

```
#include "lwip/sockets.h"
```

- 原型

```
int setsockopt(int s, int level, int optname, const void *optval, socklen_t optlen)
```

- 参数

s: socket 句柄

level/optname/optval/optlen: 参考 lwip 相应接口说明

- 返回值

小于 0 表示设置失败；0 表示设置成功

3.6.3 socket 连接

- 头文件

```
#include "lwip/sockets.h"
```

- 原型

```
int connect(int s, const struct sockaddr *name, socklen_t namelen)
```

- 参数

s: 本地 socket 句柄

name: socket 连接的目标 sockaddr 信息

namelen: sockaddr 信息长度

- 返回值

小于 0 表示连接失败；0 表示连接成功

3.6.4 socket 发送数据

- 头文件

```
#include "lwip/sockets.h"
```

- 原型

```
int send(int s, const void *data, size_t size, int flags)
```

```
int sendto(int s, const void *data, size_t size, int flags, const struct sockaddr *to, socklen_t tolen)
```

- 参数

s: 本地 socket 句柄

data: 即将发送的内容

size: 即将发送内容的大小

flags: 通常设为 0，其他值的含义请参考 lwip 相关 API 说明

to: 目标 socket 地址信息

tolen: 目标 socket 地址信息大小

- 返回值

小于 0 表示发送失败；大于 0 的值表示此次发送数据的长度

- 说明

对于 TCP 应用，使用 send()接口；对于 UDP 应用，使用 sendto()接口

3.6.5 socket 接收数据

- 头文件

```
#include "lwip/sockets.h"
```

- 原型

```
int recv(int s, void *mem, size_t len, int flags)
int recvfrom(int s, void *mem, size_t len, int flags, struct sockaddr *from,
socklen_t *fromlen)
```

- 参数

s: 本地 socket 句柄

mem: 接收内容存放的 buffer

size: 接收内容的大小

flags: 通常设为 0，其他值的含义请参考 lwip 相关 API 说明

from: 接收到的对端的地址信息

fromlen: 接收到的对端地址信息的大小

- 返回值

小于 0 表示接收失败；大于 0 的值表示此次接收数据的长度

- 说明

对于 TCP 应用，使用 recv()接口；对于 UDP 应用，使用 recvfrom()接口

3.6.6 select 监控数据

- 头文件

```
#include "lwip/sockets.h"
```

- 原型

```
int select(int maxfdp1, fd_set *readset, fd_set *writeset, fd_set *exceptset,
struct timeval *timeout)
```

- 参数

请参考 lwip 相关 API 说明

- 返回值

小于 0 表示没有数据触发；大于 0 的值表示有数据触发，UDP 场景下，此时可以调用 recvfrom()进行接收

3.6.7 关闭 socket

- 头文件

```
#include "lwip/sockets.h"
```

- 原型

```
int closesocket(int s)
```

- 参数

s: 待关闭 socket 句柄

- 返回值

小于 0 表示 close 失败；等于 0 表示 close 成功

3.6.8 TCP 带 RAI 功能的 socket 发送操作

- 头文件

```
#include "lwip/sockets.h"
```

- 原型

```
int ps_send(int s, const void *data, size_t size, int flags, u8_t dataRai,
            bool exceptdata)
```

- 参数

s: socket 句柄

data: 待发送数据指针

size: 待发送数据大小

flags: 请参考以下内容:

```
/* Flags we can use with send and recv. */
```

```
#define MSG_PEEK      0x01    /* Peeks at an incoming message */
```

```
#define MSG_WAITALL   0x02    /* Unimplemented: Requests that the function
block until the full amount of data requested can be returned */
```

```
#define MSG_OOB       0x04    /* Unimplemented: Requests out-of-band data. The
significance and semantics of out-of-band data are protocol-specific */
```

```
#define MSG_DONTWAIT   0x08    /* Nonblocking i/o for this operation only */
```

```
#define MSG_MORE       0x10    /* Sender will send more */
```

dataRai: 请参考以下定义:

```
/* SOCKET DATA RAI info: Release assistant info */
```

```
#define PS_SOCK_RAI_NO_INFO      0
```

```
#define PS_SOCK_RAI_NO_UL_DL_FOLLOWED  1
```

```
#define PS_SOCK_ONLY_DL_FOLLOWED      2
```

Exceptdata: 通常设置为 false

- 返回值

返回成功发送数据的长度

3.6.9 UDP 带 RAI 功能的 socket 发送操作

- 头文件

```
#include "lwip/sockets.h"
```

- 原型

```
int ps_sendto(int s, const void *data, size_t size, int flags,
              const struct sockaddr *to, socklen_t tolen, u8_t dataRai,
              bool exceptdata)
```

- 参数

s: socket 句柄

data: 待发送数据指针

size: 待发送数据大小

flags: 请参考以下内容:

```
/* Flags we can use with send and recv. */
```

```
#define MSG_PEEK      0x01    /* Peeks at an incoming message */
```

```
#define MSG_WAITALL   0x02    /* Unimplemented: Requests that the function
```



```
block until the full amount of data requested can be returned */
#define MSG_OOB      0x04    /* Unimplemented: Requests out-of-band data. The
significance and semantics of out-of-band data are protocol-specific */
#define MSG_DONTWAIT  0x08    /* Nonblocking i/o for this operation only */
#define MSG_MORE      0x10    /* Sender will send more */
to: 目标地址
tolen: 目标地址的长度
dataRai: 请参考以下定义:
/* SOCKET DATA RAI info: Release assistant info */
#define PS_SOCK_RAI_NO_INFO      0
#define PS_SOCK_RAI_NO_UL_DL_FOLLOWED  1
#define PS_SOCK_ONLY_DL_FOLLOWED  2
```

Exceptdata: 通常设置为 false

- **返回值**

返回成功发送数据的长度

3.7 Protocol Stack API

这一章节提供的主要是封装的底层协议栈的 API 接口，大都实现为同步接口，方便用户直接调用。

3.7.1 注册 PS 事件 callback

- **头文件**

```
#include "eventcallback.h"
```

- **原型**

```
NBStatus_t registerPSEventCallback(groupMask_t groupMask, psEventCallback_t
callback)
```

- **参数**

groupMask: 按照 group 分组过滤 Event, Group 分组定义如下:

```
typedef enum {
    NB_GROUP_BASE_MASK = (0X01 << CAC_BASE),
    NB_GROUP_DEV_MASK  = (0X01 << CAC_DEV),
    NB_GROUP_MM_MASK    = (0X01 << CAC_MM),
    NB_GROUP_PS_MASK    = (0X01 << CAC_PS),
    NB_GROUP_SIM_MASK   = (0X01 << CAC_SIM),
    NB_GROUP_SMS_MASK   = (0X01 << CAC_SMS),
    NB_GROUP_ALL_MASK   = 0X7FFFFFFF
} groupMask_t;
```

callback: 注册 PS Event 的回调函数，当有相关的事件触发时，就会在注册的回调函数中收到该信息。

目前支持的事件列表如下 (红色部分表示待实现):

事件 ID	说明
NB_URC_ID_SIM_READY	SIM 卡探测成功事件

NB_URC_ID_SIM_REMOVED	SIM 卡被移除事件
NB_URC_ID_PS_BEARER_ACTED	默认 BEARER 使能事件
NB_URC_ID_PS_BEARER_DEACTED	默认 BEARER 去使能事件
NB_URC_ID_PS_CEREG_CHANGED	网络注册状态变化事件(包含 CellID 信息)
NB_URC_ID_PS_NETINFO	网络连接状态变化事件
NB_URC_ID_MM_SIGQ	终端信号强度变化事件
NB_URC_ID_MM_TAU_EXPIRED	终端 TAU 到期通知事件
NB_URC_ID_MM_BLOCK_STATE_CHANGED	终端拥塞控制事件
NB_URC_ID_MM_NITZ_REPORT	终端网络时间事件
NB_URC_ID_MM_CERES_CHANGED	网络覆盖等级变化事件

- 返回值

NBStatusSuccess 表示注册成功; NBStatusFail 表示注册失败

3.7.2 注销 PS 事件注册

- 头文件

```
#include "eventcallback.h"
```

- 原型

```
NBStatus_t deregisterPSEventCallback(psEventCallback_t callback)
```

- 参数

Callback: 需要清除注册的 PS Event 回调函数

- 返回值

NBStatusSuccess 表示清除注册成功; NBStatusFail 表示清除注册失败

3.7.3 触发 PS 事件

- 头文件

```
#include "eventcallback.h"
```

- 原型

```
void PSTriggerEvent(urcID_t eventID, void *param, UINT32 paramLen)
```

- 参数

eventID: PS Event ID 信息

*param: 通过此 Event 传递的信息地址

ParamLen: 通过此 Event 传递的信息长度

- 返回值

无

3.7.4 获取 IMSI 号

- 头文件

```
#include "pslibapi.h"
```

- 原型

```
CmsRetId appGetImsiNumSync(CHAR *imsi)
```

- 参数

*imsi: 用于输出获得的 imsi 号, 需要传入一个地址值

- 返回值

NBStatusSuccess 表示操作成功; NBStatusFail 表示操作失败; NBStatusTimeout 表示操作超时

3.7.5 获取 ICCID 号

- 头文件

```
#include "pslibapi.h"
```

- 原型

```
CmsRetId appGetIccidNumSync (CHAR *iccid)
```

- 参数

* iccid: 用于输出获得的 iccid 号, 需要传入一个地址值

- 返回值

NBStatusSuccess 表示操作成功; NBStatusFail 表示操作失败; NBStatusTimeout 表示操作超时

3.7.6 获取 IMEI 号

- 头文件

```
#include "pslibapi.h"
```

- 原型

```
CmsRetId appGetImeiNumSync (CHAR* imei)
```

- 参数

*imei: 用于输出获得的 imei 号, 需要传入一个地址值

- 返回值

NBStatusSuccess 表示操作成功; NBStatusFail 表示操作失败; NBStatusTimeout 表示操作超时

3.7.7 获取 SN 号

- 头文件

```
#include "pslibapi.h"
```

- 原型

```
CmsRetId appGetSNumSync (CHAR* sn)
```

- 参数

* sn: 用于输出获得的 sn 号, 需要传入一个地址值

- 返回值

NBStatusSuccess 表示操作成功; NBStatusFail 表示操作失败; NBStatusTimeout 表示操作超时

3.7.8 获取激活状态 CID 信息

- 头文件

```
#include "pslibapi.h"
```

- 原型

```
CmsRetId appGetActedCidSync (UINT8 *cid, UINT8 *num)
```

- 参数

*cid: 用于输出当前处于激活状态的 CID 信息

*num: 用于输出当前处于激活状态的 CID 的个数

● 返回值

NBStatusSuccess 表示操作成功; NBStatusFail 表示操作失败; NBStatusTimeout 表示操作超时

3.7.9 获取终端 PSM 配置信息

● 头文件

```
#include "pslibapi.h"
```

● 原型

```
CmsRetId appGetPSMSettingSync(UINT8 *psmmode, UINT32 *tauTime, UINT32 *activeTime)
```

● 参数

*psmmode: 用于输出当前 PSM mode 状态, 具体内容参考下面的定义:

```
typedef enum CmiMmPsmReqModeEnum_Tag
{
    CMI_MM_DISABLE_PSM = 0,
    CMI_MM_ENABLE_PSM = 1,
    CMI_MM_DISCARD_PSM = 2 /*disable PSM, and discard PSM PARMS*/
}CmiMmPsmReqModeEnum;
```

*tauTime: 用于输出当前 PSM 信息中的 TAU 域, 单位是秒

*activeTime: 用于输出当前 PSM 信息中的 Active time 域, 单位是秒

● 返回值

NBStatusSuccess 表示操作成功; NBStatusFail 表示操作失败; NBStatusTimeout 表示操作超时

3.7.10 获取网络侧下发的 TAU 信息

● 头文件

```
#include "pslibapi.h"
```

● 原型

```
CmsRetId appGetTAUInfoSync(UINT32 *tauTime, UINT32 *activeTime)
```

● 参数

*tauTime: 用于输出网络侧下发的 PSM 信息中的 TAU 域, 单位是秒

*activeTime: 用于输出网络侧下发的 PSM 信息中的 Active time 域, 单位是秒

● 返回值

NBStatusSuccess 表示操作成功; NBStatusFail 表示操作失败; NBStatusTimeout 表示操作超时

3.7.11 修改终端 PSM 配置信息

● 头文件

```
#include "pslibapi.h"
```

● 原型

```
CmsRetId appSetPSMSettingSync(UINT8 psmmode, UINT32 tauTime, UINT32 activeTime)
```

● 参数

psmmode: 用于输入待设置的 PSM mode 状态, 具体内容参考下面的定义:

```
typedef enum CmiMmPsmReqModeEnum_Tag
{
    CMI_MM_DISABLE_PSM = 0,
    CMI_MM_ENABLE_PSM = 1,
    CMI_MM_DISCARD_PSM = 2 /*disable PSM, and discard PSM PARMS*/
}CmiMmPsmReqModeEnum;
```

tauTime: 用于输入待设置 PSM 信息中的 TAU 域, 单位是秒

activeTime: 用于输入待设置 PSM 信息中的 Active time 域, 单位是秒

● 返回值

NBStatusSuccess 表示操作成功; NBStatusFail 表示操作失败; NBStatusTimeout 表示操作超时

3.7.12 获取终端注网状态

● 头文件

```
#include "pslibapi.h"
```

● 原型

```
CmsRetId appGetCeregStateSync(UINT8 *state)
```

● 参数

*result: 用于输出获取到的当前注网状态信息

```
typedef enum CmiCeregStateEnum_Tag
{
    CMI_PS_NOT_REG = 0,
    CMI_PS_REG_HOME = 1,
    CMI_PS_NOT_REG_SEARCHING = 2,
    CMI_PS_REG_DENIED = 3,
    CMI_PS_REG_UNKNOWN = 4,
    CMI_PS_REG_ROAMING = 5,
    CMI_PS_REG_SMS_ONLY_HOME = 6, // not applicable
    CMI_PS_REG_SMS_ONLY_ROAMING = 7, // not applicable
    CMI_PS_REG_EMERGENCY = 8,
    CMI_PS_REG_CSFB_NOT_PREFER_HOME = 9, // not applicable
    CMI_PS_REG_CSFB_NOT_PREFER_ROAMING = 10 //not applicable
}CmiCeregStateEnum;
```

● 返回值

NBStatusSuccess 表示操作成功; NBStatusFail 表示操作失败

3.7.13 获取终端网络信息

● 头文件

```
#include "pslibapi.h"
```

● 原型

```
CmsRetId appGetNetInfoSync(UINT32 cid, NmAtiSyncResult *result )
```

● 参数

cid: 用于指定特定的 CID 上下文，不同的 CID 对应不同的网络信息

*result: 用于输出获取到的网络信息

其中 UE 的连接状态，可以参考 NmNetifStatus 的定义来了解具体的说明：

```

typedef enum NmNetifStatus_Tag
{
    /*
     * no netif created, and UE is not under dialing
     */
    NM_NO_NETIF_NOT_DIAL,
    /*
     * no netif created
     * 1> maybe UE is not registerd, but already under dialing,
     *    and NETIF maybe be OK later.
     * 2> or APP request one specific NETIF(by CID), which not created
     * 3> when a NETIF deactivated
     */
    NM_NO_NETIF_OR_DEACTIVATED,
    /*
     * netif is created, but NETIF is OOS
     */
    NM_NETIF_OOS,
    /*
     * netif is suspend
     * When NETIF is OOS, must also be suspend, in such case, state should be set
     * to OOS
     */
    NM_NETIF_SUSPEND,
    /*
     * netif is activated
     */
    NM_NETIF_ACTIVATED,
    /*
     * netif is activated, but some info changed, such as:
     * 1> ipv4v6 type, ipv6 RS success later, need to report NETIF changed;
     * 2> ipv4v6 two bearers, if one bearer deactivated, but another is still exist,
     *    Also need to report NETIF changed;
     * 3> This status only used for event indication report;
     */
    NM_NETIF_ACTIVATED_INFO_CHANGED
}NmNetifStatus;

```

● 返回值

NBStatusSuccess 表示操作成功；NBStatusFail 表示操作失败

3.7.14 获取基站位置信息

- 头文件

```
#include "pslibapi.h"
```

- 原型

```
CmsRetId appGetLocationInfoSync(UINT16 *tac, UINT32 *cellId)
```

- 参数

*tac: 用于输出获取的基站的 TAC 值

*cellId: 用于输出获取到的小区 ID 值

- 返回值

NBStatusSuccess 表示操作成功; NBStatusFail 表示操作失败

3.7.15 获取终端 EDRX 信息

- 头文件

```
#include "pslibapi.h"
```

- 原型

```
CmsRetId appGetEDRXSettingSync(UINT8 *actType, UINT32 *nwEdrxValueMs, UINT32 *nwPtwMs)
```

- 参数

*actType: 用于输出获取的网络侧 EDRX 配置中的 act type;

其中 0 表示没有使能 EDRX, 5 表示已经使能

*nwEdrxValueMs: 用于输出获取的网络侧 EDRX 配置中的 EDRX value 值, 单位毫秒

*nwPtwMs: 用于输出获取的网络侧 EDRX 配置中的 paging time window 值, 单位毫秒

- 返回值

NBStatusSuccess 表示操作成功; NBStatusFail 表示操作失败

3.7.16 配置终端 EDRX 信息

- 头文件

```
#include "pslibapi.h"
```

- 原型

```
CmsRetId appSetEDRXSettingSync(UINT8 modeVal, UINT8 actType, UINT32 reqEdrxValueMs)
```

- 参数

modeVal: 用于输入待设置的 EDRX 配置中的模式值, 参考以下定义:

```
typedef enum CmiMmEdrxModeEnum_Tag
{
    CMI_MM_DISABLE_EDRX = 0,
    CMI_MM_ENABLE_EDRX_AND_DISABLE_IND = 1,
    CMI_MM_ENABLE_EDRX_AND_ENABLE_IND = 2,
    CMI_MM_DISCARD_EDRX = 3 /*Disable the use of eDRX and discard
all parameters for eDRX*/
}CmiMmEdrxModeEnum;
```

actType: 用于输入待设置的 EDRX 配置中的 act type 值, 参考以下定义:

```
typedef enum CmiMmEdrxActTypeEnum_Tag
{
    CMI_MM_EDRX_NO_ACT_OR_NOT_USE_EDRX = 0,
    CMI_MM_EDRX_EC_GSM_IOT = 1, //useless
    CMI_MM_EDRX_GSM = 2, //useless
    CMI_MM_EDRX_UMTS = 3, //useless
    CMI_MM_EDRX_LTE = 4, //useless
    CMI_MM_EDRX_NB_IOT = 5
}CmiMmEdrxActTypeEnum;
```

reqEdrxValueMs: 用于输入待设置的 EDRX 配置中的 req EDRX 值,单位毫秒

- 返回值

NBStatusSuccess 表示操作成功; NBStatusFail 表示操作失败

3.7.17 获取搜网 PowerLevel 信息

- 头文件

```
#include "pslibapi.h"
```

- 原型

```
UINT8 appGetSearchPowerLevelSync(void)
```

- 参数

N/A

- 返回值

返回当前生效的搜网 PowerLevel 信息, 范围为{0, 3}

3.7.18 获取 APN 配置信息

- 头文件

```
#include "pslibapi.h"
```

- 原型

```
CmsRetId appGetAPNSettingSync(CHAR* vpn)
```

- 参数

vpn: 用于输出获取到的 vpn 信息指针, 例如传入一个数组的地址, 要求传入的空间的大小不小于 30 个字节

- 返回值

NBStatusSuccess 表示操作成功; NBStatusFail 表示操作失败

3.7.19 检查是否已经同步时间

- 头文件

```
#include "pslibapi.h"
```

- 原型

```
CmsRetId appCheckSystemTimeSync(void)
```

- 参数

- 返回值

NBStatusSuccess 表示已经获取 NITZ 时间; NBStatusFail 表示没有获取 NITZ 时间, 使用默认开始时间

3.7.20 获取当前时间

- 头文件

```
#include "pslibapi.h"
```

- 原型

```
CmsRetId appGetSystemTimeSecsSync(time_t* time)
```

- 参数

time: 用于输出获取到的时间信息指针, 类型为 time_t, 为 32 位无符号整型, 该指针输出的时间单位为秒, 起点为 1970/1/1 0 点。默认的当前时间为从 2000/1/1 的 0 点开始的时间, 等到获取到 NITZ 时间后, 会更新当前的时间

- 返回值

NBStatusSuccess 表示操作成功; NBStatusFail 表示操作失败

3.7.21 获取当前 UTC 时间

- 头文件

```
#include "pslibapi.h"
```

- 原型

```
CmsRetId appGetSystemTimeUtcSync (utc_timer_value_t *time)
```

- 参数

time: 用于输出获取到的时间信息指针, 类型为 utc_timer_value_t, 为结构体,

```
typedef __PACKED_STRUCT _utc_timer_value
{
    // UTCtimer1    ---UINT16 year--UINT8 mon--UINT8 day          //
    // UTCtimer2    ---UINT8 hour--UINT8 mins--UINT8 sec--INT8 tz  //
    unsigned int UTCtimer1;
    unsigned int UTCtimer2;
    unsigned int UTCsecs;    //secs since 1970//
    unsigned int UTCms;     //current ms //
    unsigned int timeZone;
} utc_timer_value_t;
```

默认的当前时间为从 2000/1/1 的 0 点开始的时间, 等到获取到 NITZ 时间后, 会更新当前的时间

- 返回值

NBStatusSuccess 表示操作成功; NBStatusFail 表示操作失败

3.7.22 设置当前时间

- 头文件

```
#include "pslibapi.h"
```

- 原型

```
CmsRetId appSetSystemTimeUtcSync (UINT32 time1, UINT32 time2)
```

- 参数

time1 设置年/月/日, time2 设置小时/分钟/秒/时区, 设置的时间应该从 2000/1/1 的 0 时开始
 // time1 ---UINT16 year--UINT8 mon--UINT8 day //
 // time2 ---UINT8 hour--UINT8 mins--UINT8 sec--INT8 tz //

- 返回值

NBStatusSuccess 表示操作成功; NBStatusFail 表示操作失败

3.7.23 获取终端覆盖等级信息

- 头文件

```
#include "pslibapi.h"
```

- 原型

```
UINT8 appGetCELevelSync(void)
```

- 参数

无

- 返回值

返回终端在当前小区具体的覆盖等级信息:

```
0 No Coverage Class in the serving cell
1 Coverage Class 1
2 Coverage Class 2
3 Coverage Class 3
4 Coverage Class 4
5 Coverage Class 5
```

3.7.24 获取终端信号质量信息

- 头文件

```
#include "pslibapi.h"
```

- 原型

```
CmsRetId appGetSignalInfoSync(UINT8 *csq, INT8 *snr, INT8 *rsrp)
```

- 参数

*csq: 用于输出获取的终端信号质量信息中的 csq;
 *snr: 用于输出获取的终端信号质量信息中的 snr;
 *rsrp: 用于输出获取的终端信号质量信息中的 rsrp;

- 返回值

csq 返回具体的信号强度, 取值范围为 0-31, 数值越大, 表示信号越好。一般 CSQ 值大于 5 即可正常工作。
 另外如果出现 99, 表示暂未获取到信号
 snr 返回具体的信噪比信息, SNR 越大表示信号质量越好
 rsrp 返回具体的 rsrp 信息, 如果返回 127, 表示此值无效

3.7.25 获取终端版本信息

- 头文件

```
#include "pslibapi.h"
```

- 原型

```
UINT8* appGetNBVersionInfo (void)
```

- 参数

无

- 返回值

返回软件版本号字符串信息，包含了芯片版本，以及 SDK 版本等信息

3.7.26 软复位终端 NB 功能

- 头文件

```
#include "pslibapi.h"
```

- 原型

```
CmsRetId appSetCFUN(UINT8 fun)
```

- 参数

fun: 0 表示关闭, 1 表示打开

- 返回值

NBStatusSuccess 表示操作成功; NBStatusFail 表示操作失败

3.7.27 设置启动时 NB 默认功能模式

- 头文件

```
#include "pslibapi.h"
```

- 原型

```
CmsRetId appSetBootCFUNMode(UINT8 mode)
```

- 参数

mode: 0 表示默认关闭 NB 功能, 1 表示默认打开 NB 功能

- 返回值

NBStatusSuccess 表示操作成功; NBStatusFail 表示操作失败

3.7.28 获取启动时 NB 默认功能模式

- 头文件

```
#include "pslibapi.h"
```

- 原型

```
UINT8 appGetBootCFUNMode(void)
```

- 参数

N/A

- 返回值

0 表示 NB 默认功能模式为 CFUN0; 1 表示 NB 默认功能模式为 CFUN1; 0xff 表示获取该参数失败

3.7.29 打开 SIM 逻辑通道

- 头文件

```
#include "pslibapi.h"
```

- 原型

```
CmsRetId appSetSimLogicalChannelOpenSync(UINT8 *sessionID, UINT8 *dfName)
```

- 参数

SessionID: 输出逻辑通道 ID 信息

dfName: 输入 SIM 卡应用专有文件 ID 信息

- 返回值

NBStatusSuccess 表示操作成功; NBStatusFail 表示操作失败

3.7.30 关闭 SIM 逻辑通道

- 头文件

```
#include "pslibapi.h"
```

- 原型

```
CmsRetId appSetSimLogicalChannelCloseSync(UINT8 sessionID)
```

- 参数

SessionID: 输入逻辑通道 ID 信息

- 返回值

NBStatusSuccess 表示操作成功; NBStatusFail 表示操作失败

3.7.31 SIM 逻辑通道通用访问

- 头文件

```
#include "pslibapi.h"
```

- 原型

```
CmsRetId appSetSimGenLogicalChannelAccessSync(UINT8 sessionID, UINT8 *command,  
UINT8 cmdLen, UINT8 *response, UINT8 *respLen)
```

- 参数

sessionID: 输入逻辑通道 ID 信息

command: 输入 SIM Command APDU 字符串

cmdLen: 输入 SIM Command APDU 字符串长度

response: 输出 SIM 返回的 Response APDU 字符串

respLen: 输出 SIM 返回的 Response APDU 字符串长度

- 返回值

NBStatusSuccess 表示操作成功; NBStatusFail 表示操作失败

3.7.32 SIM 通用访问

- 头文件

```
#include "pslibapi.h"
```

- 原型

```
CmsRetId appSetRestrictedSimAccessSync(CrsmCmdParam *pCmdParam,  
CrsmRspParam *pRspParam)
```

- 参数

pCmdParam: 输入参数

pRspParam: 返回参数

- 返回值

NBStatusSuccess 表示操作成功；NBStatusFail 表示操作失败

3.7.33 BAND 设置

- 头文件

```
#include "pslibapi.h"
```

- 原型

- CmsRetId appSetBandModeSync (UINT8 networkMode, UINT8 bandNum, UINT8 *orderBand)

- 参数

networkMode: 网络模式

bandNum: 设置的 band 的数量

orderBand: 设置的 band, 按顺序排放

- 返回值

NBStatusSuccess 表示操作成功；NBStatusFail 表示操作失败

3.7.34 获取当前 BAND 设置

- 头文件

```
#include "pslibapi.h"
```

- 原型

- CmsRetId appGetBandModeSync (UINT8 *networkMode, UINT8 *bandNum, UINT8 *orderBand)

- 参数

networkMode: 网络模式

bandNum: 设置的 band 的数量

orderBand: 设置的 band, 按顺序排放

- 返回值

NBStatusSuccess 表示操作成功；NBStatusFail 表示操作失败

3.7.35 获取支持的 BAND

- 头文件

```
#include "pslibapi.h"
```

- 原型

- CmsRetId appGetSupportedBandModeSync (UINT8 *networkMode, UINT8 *bandNum, UINT8 *orderBand)

- 参数

networkMode: 网络模式

bandNum: 设置的 band 的数量

orderBand: 设置的 band, 按顺序排放

- 返回值

NBStatusSuccess 表示操作成功；NBStatusFail 表示操作失败

3.7.36 获取服务小区和邻区基本信息

- 头文件

```
#include "pslibapi.h"
```

- 原型

```
CmsRetId appGetECBCInfoSync(BasicCellListInfo *bcListInfo)
```

- 参数

bcListInfo: 用于输出获取到的服务小区和邻区基本信息。

详细信息参考 BasicCellListInfo 结构定义

```
#define NCELL_INFO_CELL_NUM      4
typedef struct BasicCellListInfoTag
{
    BOOL                sCellPresent;
    UINT8               nCellNum;
    SrvCellBasicInfo    sCellInfo;
    NCellBasicInfo      nCellList[NCELL_INFO_CELL_NUM];
}BasicCellListInfo;

typedef struct SrvCellBasicInfoTag
{
    UINT16  mcc;
    UINT16  mncWithAddInfo; //if 2-digit MNC type, the 4 MSB bits should set to 'F'
    /* Example:
    * 46000; mcc = 0x0460, mnc = 0xf000
    * 00101; mcc = 0x0001, mnc = 0xf001
    * 46012; mcc = 0x0460, mnc = 0xf012
    * 460123; mcc = 0x0460, mnc = 0x0123
    */

    //DL earfcn (anchor earfcn), range 0 - 262143
    UINT32  earfcn;
    //the 28 bits Cell-Identity in SIB1, range 0 - 268435455
    UINT32  cellId;

    //physical cell ID, range 0 - 503
    UINT16  phyCellId;
    // value in dB, value range: -30 ~ 30
    BOOL    snrPresent;
    INT8    snr;

    //value in units of dBm, value range: -156 ~ -44
    INT16    rsrp;
    //value in units of dB, value range: -34 ~ 25
    INT16    rsrq;
```

```
}SrvCellBasicInfo;
```

```
typedef struct NCellBasicInfoTag
{
    UINT32      earfcn;      //DL earfcn (anchor earfcn), range 0 - 262143

    UINT16      phyCellId;
    UINT16      revd0;

    //value in units of dBm, value range: -156 ~ -44
    INT16      rsrp;
    //value in units of dB, value range: -34 ~ 25
    INT16      rsrq;
}NCellBasicInfo;
```

- 返回值

NBStatusSuccess 表示操作成功; NBStatusFail 表示操作失败

3.7.37 设置锁频, 锁小区

- 头文件

```
#include "pslibapi.h"
```

- 原型

- CmsRetId CmsRetId appSetCiotFreqSync(CiotFreqParams *pCiotFreqParams)

- 参数

pCiotFreqParams: 锁定或解锁小区 (cell), 设置或清除 prefer arfcn

```
typedef struct CiotFreqParams_Tag
{
    UINT8  mode;      // CmiDevGetFreqModeEnum 3:means UE has set preferred
                     // EARFCN list and has locked EARFCN
    UINT8  cellPresent; // indicate whether phyCellId present

    UINT16 phyCellId; // phyCell ID, 0 - 503

    UINT8  arfcnNum; // 0 is not allowed for mode is CMI_DEV_SET_PREFER_FREQ (1);
                     // max number is CMI_DEV_SUPPORT_MAX_FREQ_NUM
    UINT8  reserved0;
    UINT16 reserved1;
    UINT32 lockedArfcn; //locked EARFCN
    UINT32 arfcnList[SUPPORT_MAX_FREQ_NUM];
}CiotFreqParams;
```

- 返回值

NBStatusSuccess 表示操作成功；NBStatusFail 表示操作失败

3.7.38 查询锁频，锁小区状态

- 头文件

```
#include "pslibapi.h"
```

- 原型

- `CmsRetId CmsRetId appGetCiotFreqSync(CiotFreqParams *pCiotFreqParams)`

- 参数

pCiotFreqParams: 获取到的参数，参考 CiotFreqParams 结构体

- 返回值

NBStatusSuccess 表示操作成功；NBStatusFail 表示操作失败

4. 附录

4.1 上电启动流程

如下图，在 SDK 的上电启动过程中，黑色部分是 SDK 会自动执行的流程，红色部分是用户可以参与的部分，通过 Event 注册机制或者直接 API 调用来实现。

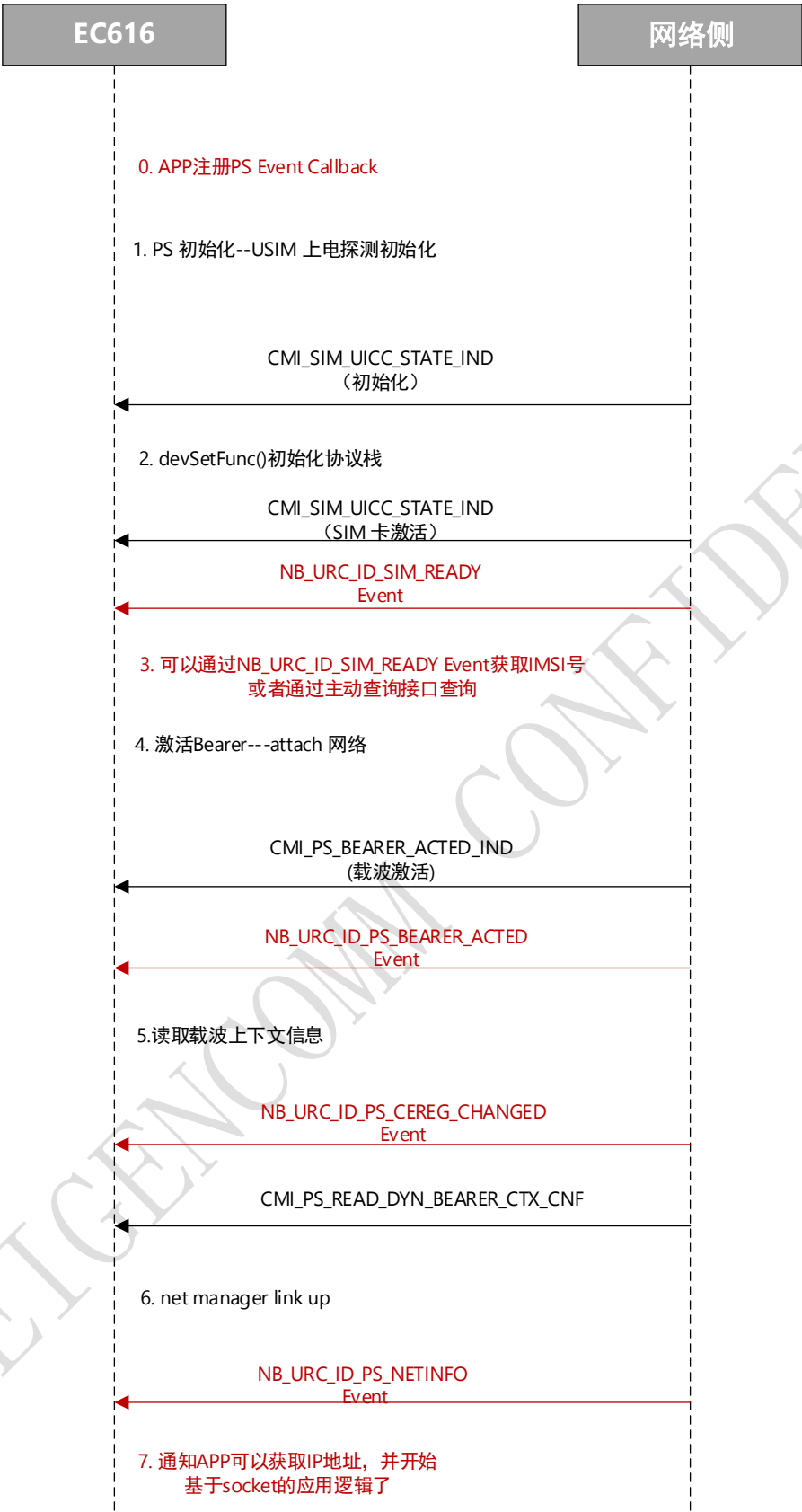


图 3 启动流程

4.2 OpenCPU 应用示例

SDK 中 project/\$(TARGET)/apps 目录下包含了以下 OpenCPU 的应用示例供用户参考：

示例	说明
coap_example	如何使用 libcoap 开源软件的示例
driver_example	驱动示例
fs_example	Littlefs 文件系统示例
https_example	https 示例
lwm2m_example	wakaama lwm2m 使用示例
mqtt_example	MQTT 使用示例
onenet_example	连接 onenet 平台的睡眠唤醒示例
oc_example	连接电信 OC 平台的睡眠唤醒示例
socket_example	Tcp/udp client/server 使用示例

5. 版本

版本	日期	备注
Draft	2018-11-11	
1.0	2018-12-01	增加 log API 描述
1.1	2020-04-02	修改描述，支持 NB-IoT SDK（EC616/617） 增加 PS API
1.2	2020-08-06	文件系统 API 从 LFS 换成 OSA
1.3	2020-08-12	更新用户预留 RAM 空间和使用说明
1.3	2020-08-18	增加 OsaFsync、OsaFtell 接口

6. 关于我们

上海移芯通信科技有限公司坐落于上海张江，公司于 2017 年 2 月成立，从事蜂窝移动通信芯片的研发和销售。公司产品主要应用于广域物联网通信领域，致力于设计最具性价比的蜂窝物联网芯片。公司团队在蜂窝通信芯片上有着辉煌历史和丰富经验。公司所有核心技术全部自研，包括算法&架构、射频、基带、SoC、协议栈软件、平台&应用软件和硬件方案等。

移芯通信首款自主研发的超低功耗 NB-IoT 芯片 EC616 于 2019 年中量产，被绝大部分头部模组企业采用，现已大批量应用于全国各区域各行业。下一代超高集成度 NB-IoT 芯片 EC616S 于 20 年底量产，超低功耗 4G Cat.1 芯片 EC618 工程样片阶段，5G 芯片正在研发中。

上海移芯通信科技有限公司

地址：中国上海市浦东新区祥科路 298 号佑越国际 1 幢 7 层

邮编：201210

电话：

电邮：

网址：<http://www.eigencomm.com>

技术支持窗口

电邮：support@eigencomm.com